

CHAPTER 2 · 보충

모형 평가와 모형 선택 Train/Test 분할 · AIC · Stepwise

왜 모형을 만든 데이터와 평가하는 데이터를 나눠야 할까? 그리고 선형회귀에서 “좋은 모형”은 어떻게 찾을까?

Part 2 의 `lm()` · `step()` 실습과 이어지는 개념 정리

왜 데이터를 나눠서 평가해야 할까 ?

모형의 진짜 목적은 이미 본 데이터를 설명하는 것이 아니라 , 아직 보지 못한 새 데이터를 예측하는 것

같은 데이터로 적합 + 평가

- 기출문제로 공부하고 , 똑같은 기출문제로 시험 보는 것
- 답을 “외운” 모형도 만점 → 실력이 과대평가됨
- 모형이 복잡할수록 이 착시는 더 커짐

Training 으로 적합 , Test 로 평가

- 공부에 쓰지 않은 새 문제로 시험 보는 것
- 외운 것이 아니라 “일반화” 된 실력을 측정
- Test error = 새 데이터에서의 예측 오차의 추정치

Training error 는 항상 낙관적이다 . 모형 평가는 반드시 적합에 쓰지 않은 데이터로 !

Train/Test 분할의 기본 절차

전체 데이터 (예 : clean.algae 198 개)

Training set (약 70%) → 모형 적합

Test set (약 30%) → 평가

1

무작위로 나누기

데이터를 섞은 뒤 일부를 test set 으로 떼어 둔다 (적합 과정에서 절대 사용 금지)

2

Training set 으로 적합

모형의 계수 추정 , 변수 선택 등 모든 “학습” 은 training set 에서만

3

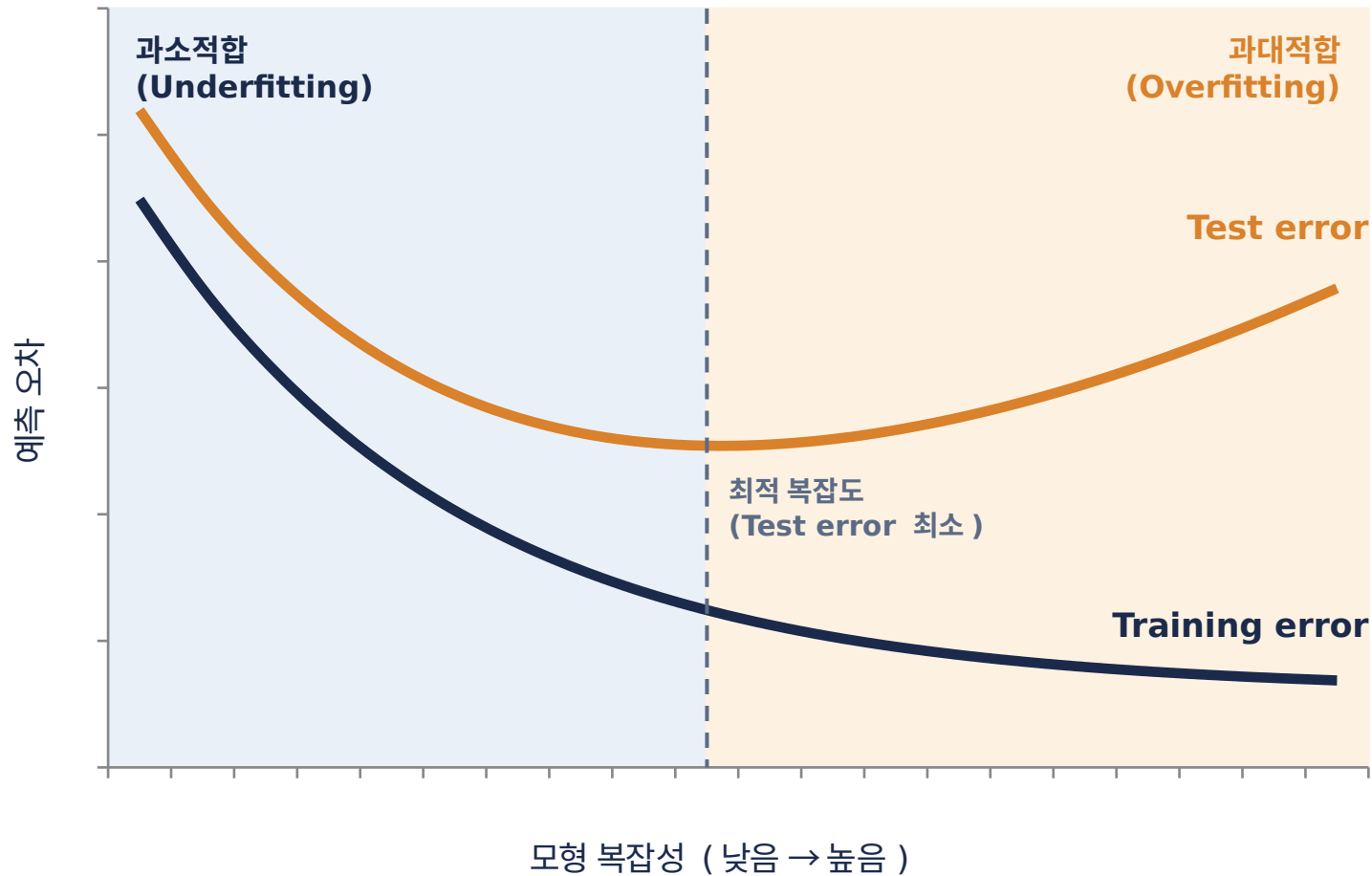
Test set 으로 평가

적합된 모형으로 test set 을 예측하고 MSE·MAE·NMSE 등 오차를 계산

R CONSOLE

```
> set.seed(1234)
> idx <- sample(1:198, size=139)
> train <- clean.algae[idx, 1:12]
> test <- clean.algae[-idx, 1:12]
> m <- lm(a1 ~ ., data=train)
> pred <- predict(m, test)
> mean((pred - test$a1)^2) # test MSE
```

모형 복잡성과 예측 오차



Training error

복잡도가 높아질수록 계속 감소. 충분히 복잡하면 0에 가까워짐

Test error

처음엔 감소하다가, 어느 지점 이후 다시 증가 (U자 모양)

두 곡선의 간격

복잡할수록 벌어짐 = 학습 데이터의 “잡음” 까지 외운 정도

과소적합 vs 적정 vs 과대적합

과소적합

Underfitting

- 모형이 너무 단순
- 중요한 패턴을 놓침 (편향↑)
- Training·Test error 모두 큼
- 예 : 곡선 관계를 직선 하나로

적정 복잡도

Good fit

- 데이터의 구조는 잡고 잡음은 무시
- Test error 가 가장 작음
- 새 데이터에도 안정적으로 예측
- 우리가 찾는 목표 모형

과대적합

Overfitting

- 모형이 너무 복잡
- 잡음까지 학습 (분산↑)
- Training error 매우 작음, Test error 큼
- 예 : 변수를 모두 넣은 회귀, 깊은 나무

편향 (bias) 과 분산 (variance) 의 균형 : 복잡도를 높이면 편향은 줄고 분산은 커진다 → 그 균형점이 test error 최소 지점

선형회귀에서 “복잡성”이란 ?

- 선형회귀에서 모형의 복잡성 = 모형에 포함된 설명변수 (모수) 의 개수
- 변수를 하나 추가하면 RSS (잔차제곱합) 는 절대 늘지 않고 , R^2 는 절대 줄지 않음
- 따라서 training error(RSS, R^2) 로 모형을 고르면 항상 “변수를 모두 넣은 모형” 이 선택됨 → 과대적합 위험

Test error 가 최소인 모형을 찾는 두 가지 방법

1 직접 측정

Test set 또는 교차검증 (CV) 으로 후보 모형들의 예측 오차를 직접 계산해 비교

2 정보 기준으로 근사 : AIC

Training 데이터만으로 계산하되 , 복잡도에 벌점을 줘서 test error 를 근사 → 선형회귀에서는 AIC 가 최소인 모형을 찾는 것이 목표

AIC (Akaike Information Criterion) 의 정의

$$AIC = -2 \cdot \log L(\theta) + 2 \cdot k$$

$-2 \cdot \log L(\hat{\theta})$: 적합도 항

최대우도 $L(\hat{\theta})$ — 모형이 데이터를 얼마나 잘 설명하는가. 잘 맞을수록 작아짐 (training error 역할)

$2 \cdot k$: 복잡도 벌점 항

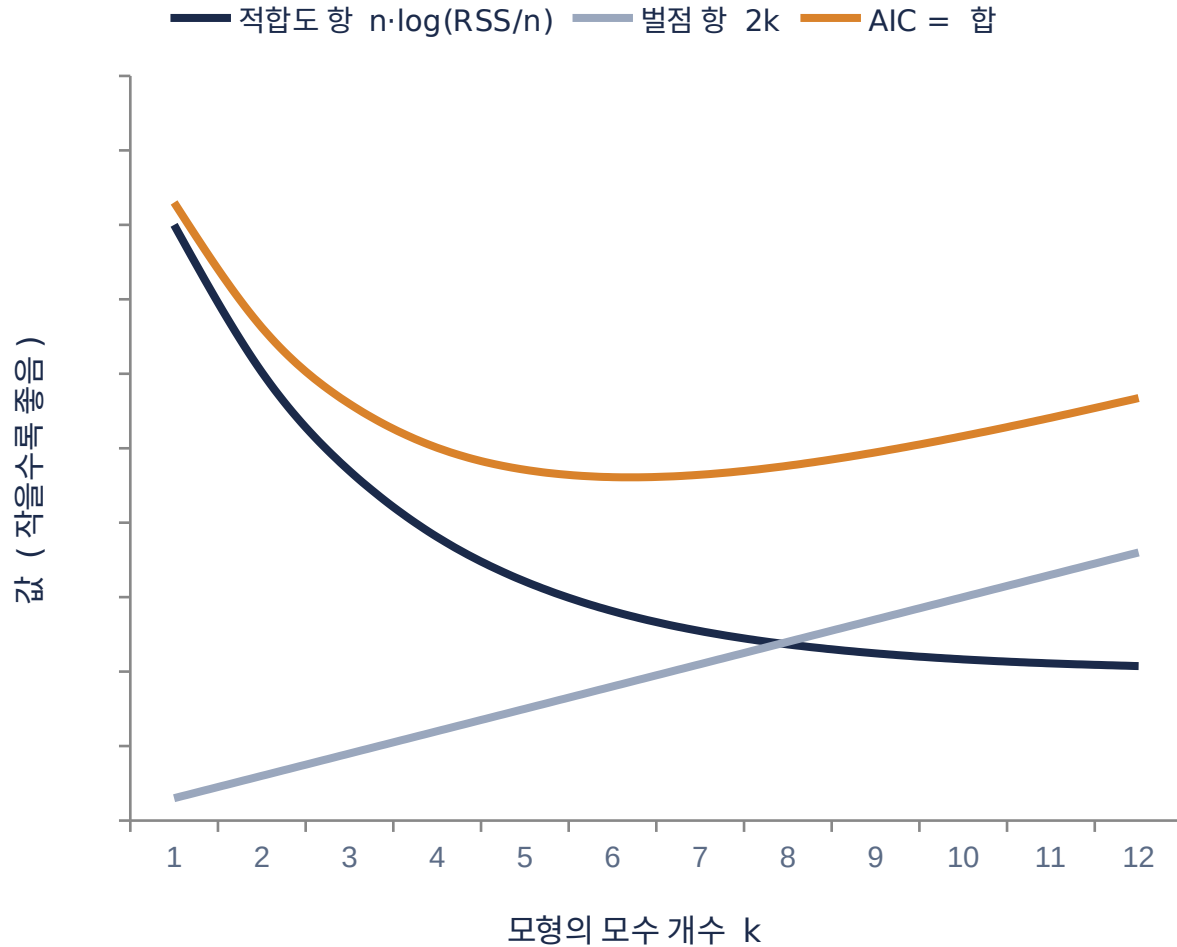
k = 추정하는 모수의 개수 (절편 포함 회귀계수 수). 모수가 하나 늘 때마다 벌점 +2

정규오차 선형회귀에서는

$$AIC = n \cdot \log(RSS / n) + 2k (+ 상수)$$

n : 관측치 수, RSS : 잔차제곱합
상수는 모든 모형에 같으므로 비교에 영향 없음
(R의 `step()` 은 이 형태 `extractAIC()` 를 사용)

AIC 는 무엇을 뜻하나 ?



1

작을수록 좋은 모형

후보 모형 중 AIC 가 가장 작은 모형을 선택

2

적합도와 단순성의 균형

변수를 추가하려면 로그우도를 1 이상 올려야 “본전” (벌점 +2 상쇄)

3

Test error 의 근사

새 데이터에 대한 기대 예측 오차 (KL 거리) 의 추정치 — 데이터를 나누지 않고도 U 자 곡선의 최소점을 겨냥

4

상대 비교만 의미

값 자체보다 모형 간 차이가 중요 . 같은 데이터 · 같은 반응변수에서만 비교 가능

AIC 최소 모형을 어떻게 찾을까 ?

설명변수가 p 개이면 , 가능한 부분집합 모형은 2^p 개 $\rightarrow$ 모두 적합해 보는 것 (전역 탐색) 은 금방 불가능해진다

$p = 11$

2,048

Part 2 algae 예제의 변수 수

$p = 20$

약 105 만

이미 매우 부담스러운 계산

$p = 30$

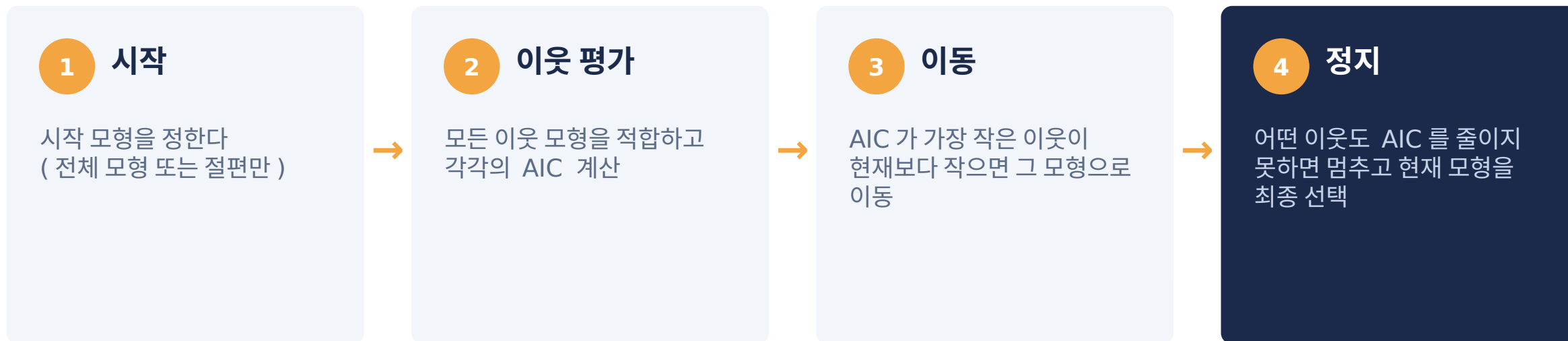
약 10.7 억

전역 탐색은 사실상 불가능

해결책 : 국소 탐색 (local search). 전체를 다 보지 않고 , 현재 모형 “주변” 만 살펴보며 AIC 가 줄어드는 방향으로 한 걸음씩 이동 $\rightarrow$ 이것이 stepwise 선택

Stepwise: 일종의 국소 탐색 (local search)

이웃 (neighbor) = 현재 모형에서 변수 하나를 추가하거나 제거한 모형

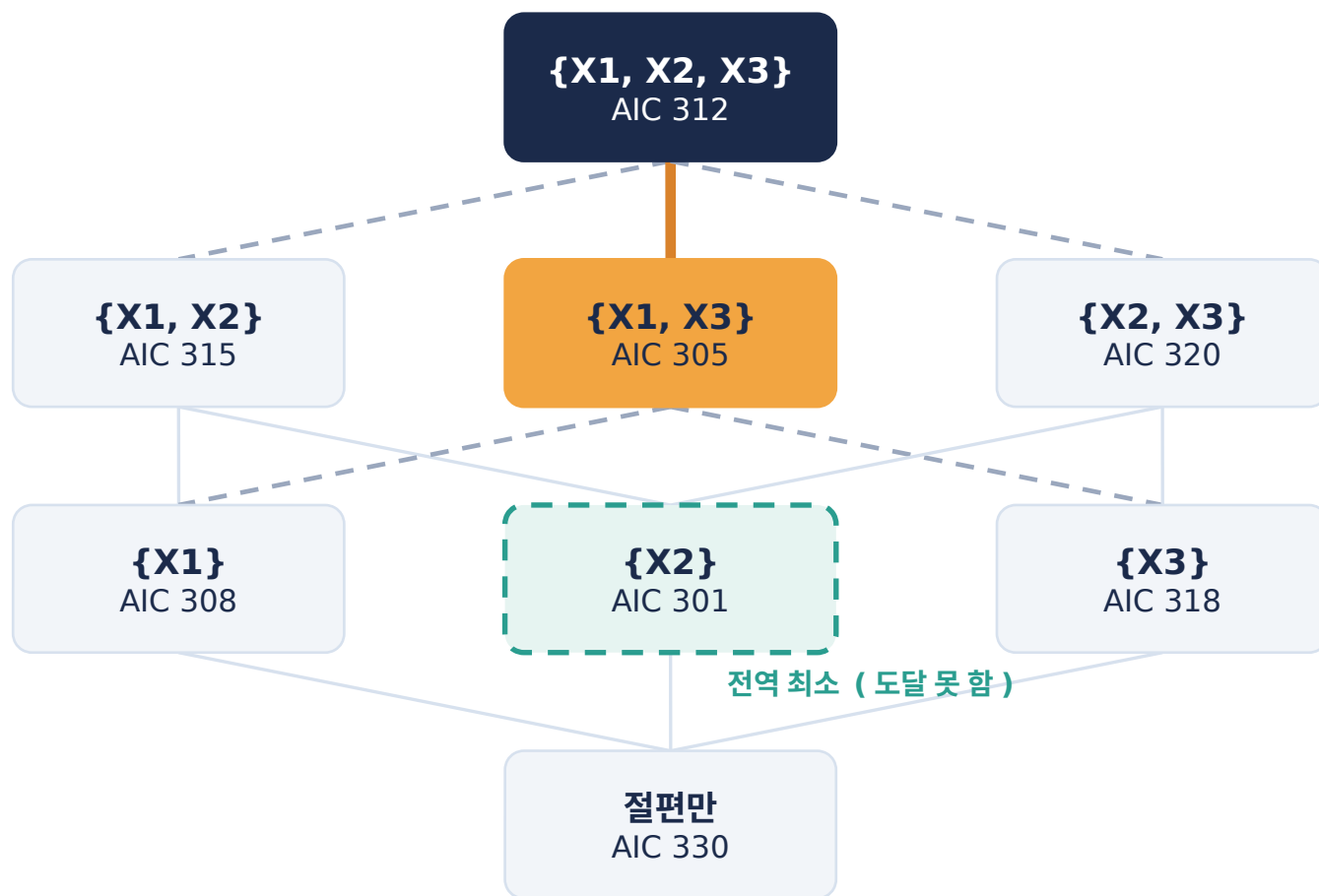


↻ 2-3 단계를 개선이 없을 때까지 반복

비유 : 안개 낀 산에서 가장 낮은 골짜기를 찾을 때 , 산 전체를 볼 수 없으니 발밑에서 가장 가파르게 내려가는 쪽으로 한 걸음씩 걷는 것 (탐욕적 ·greedy 탐색)

EXAMPLE

예시 : 변수 3 개일 때의 stepwise 경로 (후진 제거)



* AIC 값은 설명을 위한 가상의 예시

Step 0 전체 모형 $\{X1, X2, X3\}$ 에서 시작 · AIC 312

Step 1 이웃 3 개 비교

– $X1 \rightarrow 320$, – $X2 \rightarrow 305$, – $X3 \rightarrow 315$

→ 가장 작은 $\{X1, X3\}$ (305) 로 이동

Step 2 이웃 비교

– $X1 \rightarrow 318$, – $X3 \rightarrow 308$, + $X2 \rightarrow 312$

→ 모두 305 보다 큼 → 정지

결과 최종 $\{X1, X3\}$ (305). 하지만 $\{X2\}$ (301) 가 더 좋은데도 한 걸음 이웃이 아니어서 찾지 못함 = 국소 최적 (local optimum)

Stepwise 의 세 가지 방향과 R 의 step()

전진 선택

Forward

절편만 있는 모형에서 시작, 변수를 하나씩 추가만

후진 제거

Backward

전체 모형에서 시작, 변수를 하나씩 제거만

단계적 선택

Both

매 단계마다 추가와 제거를 모두 이웃으로 고려 (탐색 범위가 가장 넓음)

R CONSOLE

```
# 후진 제거 (Part 2 에서 사용)
> final.lm <- step(lm.a1)

# 전진 선택
> null.lm <- lm(a1 ~ 1, data=clean.algae[,1:12])
> step(null.lm, scope=formula(lm.a1),
+       direction="forward")

# 단계적 선택
> step(null.lm, scope=formula(lm.a1),
+       direction="both")
```

Part 2 결과 : step(lm.a1) 은 11 개 변수에서 시작해 AIC 를 줄이며 size, mxPH, Cl, NO3, PO4 5 개 변수 모형에서 멈췄다

Stepwise 사용 시 주의할 점

1 국소 최적

전역 최소를 보장하지 않음 . 시작점 · 방향에 따라 다른 모형에 도착할 수 있음

2 AIC 도 “추정치”

AIC 는 test error 의 근사일 뿐 . 표본이 작거나 변수가 많으면 근사가 부정확해짐

3 선택 후 p-value 는 낙관적

데이터로 변수를 고른 뒤 같은 데이터로 한 검정은 유의성을 과대평가하기 쉬움

4 최종 확인은 test set 으로

stepwise 는 training set 에서만 수행하고 , 선택된 모형의 성능은 test set (또는 CV) 으로 평가

흐름 정리 : Training set 에서 AIC + stepwise 로 후보 모형 선택 → Test set 에서 예측 오차로 최종 검증

SUMMARY

오늘의 핵심 정리

1

Train/Test 분할

Training error 는 복잡할수록 계속 감소하지만 , test error 는 U 자 . 평가는 적합에 쓰지 않은 데이터로 .

2

$AIC = -2 \log L + 2k$

적합도 + 복잡도 벌점 . 선형회귀에서는 AIC 가 최소인 모델을 찾는 것이 목표 .

3

Stepwise = 국소 탐색

변수 하나 추가 / 제거한 이웃 중 AIC 가 가장 작은 쪽으로 이동 . 빠르지만 국소 최적 가능 .